

Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with each other. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Task 1

As we can see, the CPU matrix implementation is pretty slow, as the operations are done sequentially. However, this implementation gives a baseline result, we can use to compare with the GPU kernels' results.

Task 2

For task 2 we should create a baseline speed, we can use to compare the effective bandwidths of the different implementations. For the row2row copy kernel, the most basic memory operation is performed. We see, that the row2row copy has a bandwidth of nearly 1234 GB/s. Compared with the results from the lecture, the memory bandwidth is significantly higher.

Task 3

In task 3 the naive transpose kernel should be implemented and evaluated. In the lecture, we see that the naive transpose has a third to a fifth of the effective bandwidth which was observed for the copy benchmark. Same can be observed here. The bandwidth is about one quarter of the observed bandwidth of the row2row copy.

This is because the memory access for transposing the kernel is not optimal at all. While for the copy kernel, all threads of a warp access elements from the memory which lay next to each other ($y * \text{width} + 0, 1, 2, 3, \dots$), the memory access is optimized. For the transpose, each thread of a warp access a single 4-byte float on its own as the memory addresses are far from another.

Task 4

In task 4 we try to fix the issue observed in task 3. Before, we had inefficient memory accesses. To fix this, shared memory is used. In the lecture, there was a distinction between padded and not padded shared memory as the padded memory avoids bank conflicts. Same is done here. There is one kernel which has padded memory while the other one doesn't.

In the lecture, we see, that using shared memory speeds up the kernel. Same can be seen in our benchmarks. With shared memory and no padding we are twice as fast as the naive transpose kernel which matches the results from the lecture. This is because the data is only loaded once into the shared memory and the actual operation can be performed on the faster shared memory.

To even improve the performance, padding can be used as this avoids bank conflicts. Using padding, the kernel is about three times as fast as without padding, showing that bank conflicts are also slowing down the memory access. The effective bandwidth here is ~1744 GB/s, which is faster than the row2row copy bandwidth reported above. This is because even if the memory addresses lie next to each other, it is faster to load the data into the shared memory.